Bangladesh University of Engineering & Technology

Course: CSE-462 (Algorithm Engineering Sessional)
Project: Multidimensional Knapsack Problem

Omega Group 9

Submitted by:
Shovon Roy (2005010)
Tusher Bhomik (2005046)
Md. Rabby Hossain (2005053)
Souvik Mandol (2005069)
Sagor Chanda (2005071)
Iffat Bin Hossain (2005087)

Date of submission: April 10, 2026

Abstract

The Multidimensional Knapsack Problem (MKP) is a strongly NP-Hard combinatorial optimization problem with applications in resource allocation, capital budgeting, and project selection. Given the computational intractability of exact methods for large instances, efficient heuristic algorithms are essential. This report presents a comprehensive experimental comparison of four constructive heuristics: Toyoda, Improved Toyoda, Randomized Greedy, and Improved Randomized Greedy. We evaluate all algorithms on 344 benchmark instances from the OR-Library spanning varying problem sizes ($n \in \{100, 250, 500\}$) and constraint tightness levels ($\alpha \in \{0.25, 0.50, 0.75\}$). Results demonstrate that Improved Randomized Greedy achieves the best solution quality, winning 56.7% of instances, with a mean profit improvement of 0.3% over Toyoda at the cost of 8.5× runtime overhead. Improved Toyoda provides the best deterministic baseline, while Randomized Greedy offers an effective balance between quality and efficiency. Our findings provide practical guidelines for algorithm selection based on problem characteristics and computational budget.

Contents

1 Introduction

1.1 Problem Definition

The Multidimensional Knapsack Problem (MKP) is a fundamental combinatorial optimization problem that extends the classical 0/1 knapsack problem to multiple resource constraints. Formally, MKP is defined as follows:

$$\begin{aligned} \text{Maximize} \quad & Z = \sum_{j=1}^{n} p_j x_j \\ \text{Subject to} \quad & \sum_{j=1}^{n} w_{ij} x_j \leq C_i, \quad i = 1, \ldots, m \\ & x_j \in \{0, 1\}, \quad j = 1, \ldots, n \end{aligned} \tag{1}$$

where:

- n is the number of items
- m is the number of resource constraints (dimensions)
- p_j is the profit of item j
- w_{ij} is the weight of item j with respect to constraint i
- C_i is the capacity of constraint i
- x_j is a binary decision variable indicating whether item j is selected

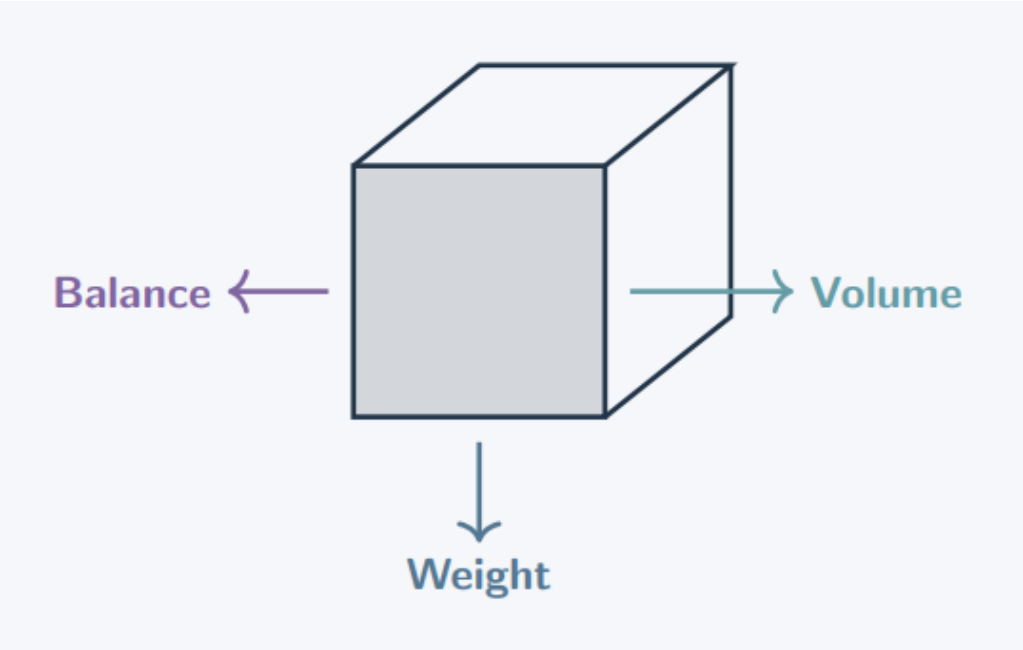

Figure 1: The "Cargo Plane" analogy for MKP. Unlike a standard knapsack where only weight matters, MKP requires simultaneously satisfying multiple constraints: maximum takeoff weight (W), cargo bay volume (V), and center of gravity/balance (B). The formal mathematical formulation $Z = \sum p_j x_j$ subject to $\sum w_{ij} x_j \leq C_i$ captures this multi-constraint nature.

Unlike the single-constraint knapsack problem, MKP requires that *all* m constraints be satisfied simultaneously, making it significantly more challenging. The problem is strongly NP-Hard, proven via polynomial-time reduction from the standard knapsack problem [4, 5].

1.2 Why is MKP Harder than Standard Knapsack?

The key challenge in MKP arises from the "hidden constraint" problem. In the standard knapsack, we only check one dimension (weight). In MKP, an item might look "good" in one dimension but "break" another constraint entirely.

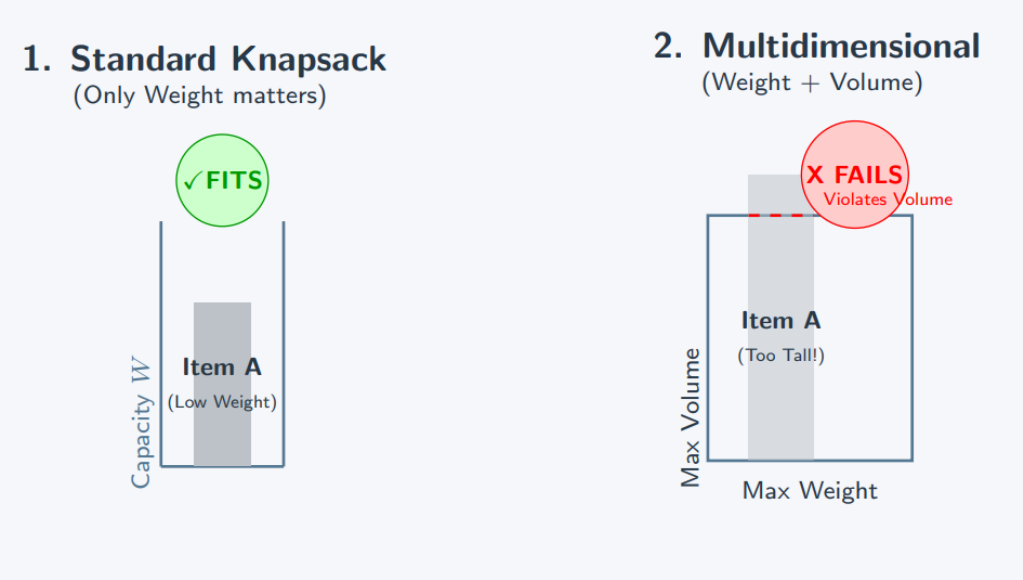

Figure 2: Visual comparison between Standard Knapsack and Multidimensional Knapsack. In the standard case (left), Item A fits because only weight is checked. In the multidimensional case (right), the same Item A fails because it violates the volume constraint despite fitting the weight constraint. This illustrates why MKP is fundamentally harder—feasibility must be verified across *all* dimensions simultaneously.

1.3 Motivation and Applications

MKP has numerous real-world applications including:

- **Resource Allocation:** Assigning limited resources (budget, time, personnel) to projects with multiple constraints
- **Capital Budgeting:** Selecting investment portfolios subject to risk, liquidity, and diversification constraints
- **Cargo Loading:** Optimizing container loading with weight, volume, and balance restrictions
- **Task Scheduling:** Allocating tasks to processors with CPU, memory, and bandwidth limits

The computational complexity of MKP grows exponentially with problem size. While exact methods such as branch-and-bound and integer linear programming can solve small instances optimally, they become impractical for large-scale problems ($n > 100$). This motivates the development of efficient heuristic algorithms that can produce high-quality solutions in reasonable time.

1.4 Illustrative Example

To build intuition, consider a small instance with 4 items and 2 dimensions (Weight & Volume):

Table 1: Illustrative MKP instance: 4 items, 2 dimensions

Item	**Profit** p_j	**Weight** w_{1j}	**Volume** w_{2j}
x_1	10	4	3
x_2	6	2	5
x_3	8	3	2
x_4	5	1	4

Capacity constraints: $C_1 = 6$ (weight), $C_2 = 7$ (volume).

First attempt (infeasible): Selecting x_1 and x_3 yields profit $10+8 = 18$, but weight $= 4 + 3 = 7 > 6$, violating the weight constraint.

Feasible optimal solution: Selecting x_3 and x_4: Weight $= 3+1 = 4 \leq 6$ ✓, Volume $= 2 + 4 = 6 \leq 7$ ✓. Optimal profit $= 8 + 5 = 13$.

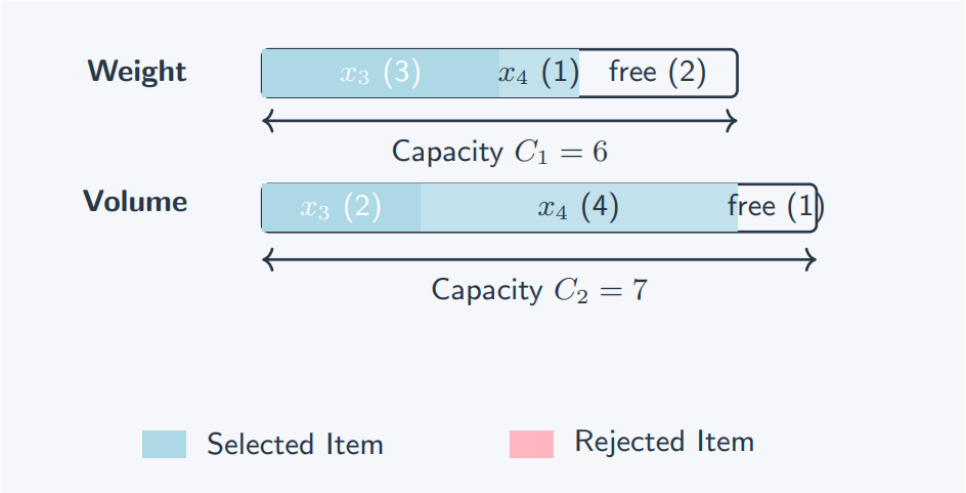

Figure 3: Capacity utilization visualization for the optimal solution (items x_3 and x_4 selected, profit = 13). The weight dimension uses 4 out of 6 units (67% utilization) while the volume dimension uses 6 out of 7 units (86% utilization). The remaining slack in each dimension represents potential capacity that no remaining feasible item can exploit.

1.5 Research Questions

This study addresses the following research questions:

RQ1: Does the fill-up pass in Improved Toyoda consistently improve profit over Toyoda?

RQ2: Does Randomized Greedy outperform both deterministic Toyoda variants across different problem sizes and tightness levels?

RQ3: Does the elite seeding mechanism in Improved Randomized Greedy improve solution quality compared to standard randomized greedy?

RQ4: How does constraint tightness (α) affect algorithm performance and variance?

1.6 Report Organization

The remainder of this report is organized as follows: Section 2 presents the complexity analysis and reductions. Section 3 surveys existing algorithms. Section 4 describes the four algorithms with complexity analysis and pseudocode. Section 5 details the experimental setup including datasets, metrics, and methodology. Section 6 presents comprehensive performance results across all 344 benchmark instances. Section 7 interprets the findings and answers the research questions. Section 8 concludes with contributions and future work.

2 Complexity Analysis and Reductions

2.1 The Complexity Hierarchy

To understand the hardness of MKP, we locate it between two well-known problems using polynomial-time reductions. MKP sits in a "sandwich" between the Standard Knapsack Problem (weakly NP-Hard) and general 0/1 Integer Linear Programming (NP-Hard).

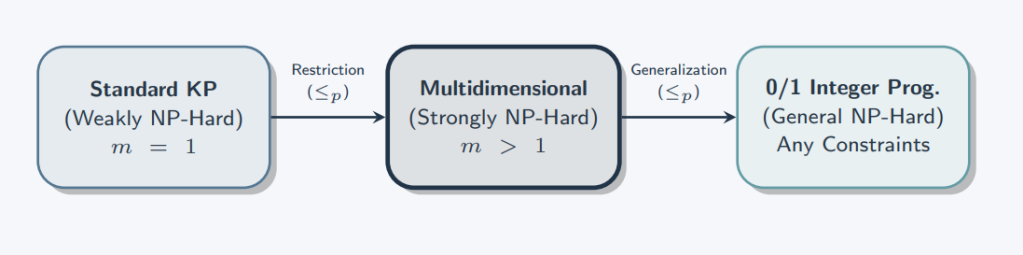

Figure 4: The "Sandwich" hierarchy of computational complexity. Standard Knapsack ($m = 1$, weakly NP-Hard) reduces to MKP ($m > 1$, strongly NP-Hard) via restriction, proving MKP inherits NP-Hardness. MKP in turn generalizes to 0/1 Integer Linear Programming (general NP-Hard), proving MKP is solvable by ILP solvers. The reduction from left proves hardness; the reduction to right proves solvability.

2.2 Reduction from Standard Knapsack (KP $\leq_p$ MKP)

The standard knapsack problem is a special case of MKP with $m = 1$. Given any KP instance with items n, weights w_j, and capacity C, we construct an MKP instance by setting $m = 1$, $C_1 = C$, and $w_{1j} = w_j$. Since KP is NP-Hard and is embedded inside MKP, MKP must be *at least* as hard.

2.3 Generalization to ILP (MKP $\leq_p$ ILP)

MKP constraints are a specific type of linear constraint. The translation to standard matrix format $\mathbf{Ax} \leq \mathbf{b}$ is direct:

$$\underbrace{\begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,n} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \cdots & w_{m,n} \end{bmatrix}}_{\mathbf{A}\ (m\times n)} \times \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\mathbf{x}\ (n\times 1)} \leq \underbrace{\begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_m \end{bmatrix}}_{\mathbf{b}\ (m\times 1)} \tag{2}$$

This proves MKP is solvable by any standard Branch-and-Cut algorithm used in commercial ILP solvers (e.g., CPLEX, Gurobi).

3 Existing Algorithm Categories

The Multidimensional Knapsack Problem (MKP) has been studied extensively because it captures the essential difficulty of making binary selection decisions under several simultaneous resource constraints. Unlike the classical knapsack problem, where a single capacity restriction must be respected, MKP requires every chosen item to remain feasible across multiple dimensions at the same time. This additional coupling among constraints makes the problem significantly harder and has motivated a broad spectrum of solution strategies. In the literature, these strategies are typically classified into five major categories: exact algorithms, approximation algorithms, heuristic algorithms, metaheuristic algorithms, and randomized algorithms. Each category reflects a different balance between optimality, computational cost, and scalability, and each has its own role in both theoretical and practical optimization. [9, 11]

3.1 Exact Algorithms

Exact algorithms are designed to compute an optimal feasible solution, and therefore they provide the strongest possible guarantee of correctness. Their appeal lies in the fact that they do not rely on empirical assumptions or approximation quality; instead, they systematically explore the feasible region until the optimal value is certified. However, this guarantee comes at a considerable computational cost. For MKP, the search space grows exponentially with the number of items, and the presence of multiple capacity constraints makes pruning and state reduction substantially more difficult than in the one-dimensional case. As a result, exact methods are typically practical only for small or moderately sized instances, or for benchmark settings where optimality is the primary objective. [11, 10]

Among the most widely used exact approaches are branch-and-bound, branch-and-cut, dynamic programming, and integer linear programming formulations. Branch-and-bound organizes the solution space as a search tree and uses upper bounds to eliminate branches that cannot improve upon the best incumbent solution. In MKP, such bounds are often derived from linear relaxations, which make the method significantly more effective than a brute-force enumeration, yet still expensive in the worst case. Branch-and-cut strengthens this framework by introducing valid inequalities that tighten the

relaxation and reduce the number of nodes that must be explored. This makes branch-and-cut one of the most powerful exact paradigms for structured combinatorial problems. [1, 6]

Dynamic programming provides another exact route by extending the classical knapsack recurrence to multiple dimensions. In principle, this yields a complete solution method, but the state space grows rapidly with the number of constraints and the size of the capacities. Consequently, the memory and time requirements can become prohibitive even for moderately sized instances. For this reason, dynamic programming is mainly useful as a conceptual foundation or as a subroutine inside more advanced exact methods, rather than as a general-purpose solver for large MKP instances. [10]

ILP-based approaches formulate MKP directly as a binary optimization model and delegate the solution to a general-purpose solver. This formulation is attractive because MKP naturally fits the ILP framework: the objective is linear, the constraints are linear, and the decision variables are binary. Modern solvers combine branching, cutting planes, presolve, and bound tightening, which can make them highly competitive in practice. Nevertheless, their running time can still rise sharply on difficult instances, especially when the number of dimensions is large or the constraints are tight. [11]

3.2 Approximation Algorithms

Approximation algorithms occupy a middle ground between exactness and efficiency. Their objective is not to guarantee the optimal solution, but rather to produce a feasible solution whose quality can be formally bounded relative to the optimum. This makes them especially useful when the instance size is too large for exact methods, but a theoretical performance guarantee is still desired. In MKP, approximation algorithms are valuable from both a methodological and a practical perspective, since they reveal how close one can get to the optimal solution in polynomial time under controlled assumptions. [2, 7]

A common approximation strategy is linear programming relaxation followed by rounding. The binary restrictions are relaxed so that variables may take fractional values, the relaxed problem is solved efficiently, and the fractional values are then converted into an integral feasible solution. Because simple rounding can violate one or more constraints, a repair phase is usually required to restore feasibility. The quality of the final solution depends on how carefully the rounding and repair steps are designed, and on how well the LP relaxation captures the structure of the original instance. [13]

For MKP, approximation schemes are often discussed under the assumption that the number of dimensions is fixed. In that setting, polynomial-time approximation schemes can be constructed by enumerating a small number of important items exactly and then filling the remaining capacity greedily or via relaxation-based methods. These schemes are theoretically important because they show that near-optimal solutions can be obtained efficiently when the dimensionality is controlled. At the same time, the practical utility of such methods decreases as the number of constraints increases, since the enumeration stage becomes more expensive and the structure of the problem becomes less favorable. [10, 7]

3.3 Heuristic Algorithms

Heuristic algorithms are among the most widely used methods for MKP in practice because they offer a strong balance between simplicity, speed, and solution quality. Unlike exact and approximation algorithms, heuristics do not aim to provide a formal optimality guarantee. Instead, they rely on problem-specific insight to construct a good feasible solution quickly. This makes them especially attractive for large instances, real-time decision systems, and applications where a near-optimal answer is sufficient. Heuristics also serve as an important baseline for evaluating more advanced methods, since they reveal how much improvement can be obtained from additional search effort. [15, 12]

Constructive heuristics build a solution step by step from an initially empty or partially defined state. The classical Toyoda-type greedy approach is one of the best-known examples in knapsack literature. It selects items according to a profit-to-consumption criterion, thereby favoring candidates that provide high reward while using relatively little resource. In the multidimensional setting, this idea must be adapted so that the selection rule reflects the interaction among all capacities rather than only one constraint. The main advantage of such methods is their very low computational cost and their ease of implementation. Their main weakness is that early greedy decisions may later block access to better combinations of items. [15]

Dual greedy heuristics follow the opposite direction. Rather than starting from an empty knapsack, they begin with a full or nearly full selection and iteratively remove items until feasibility is restored. This strategy can be useful when a constructive greedy method leaves too much capacity unused or gets trapped in a poor local configuration. By viewing feasibility from both directions, primal and dual greedy methods often complement one another and provide a more complete baseline for MKP experimentation. [12]

Local-improvement heuristics attempt to refine an initial feasible solution by exploring nearby configurations. Typical operations include exchanging selected and unselected items, removing one item and inserting another, or performing small multi-item swaps that preserve feasibility. These methods are particularly valuable for MKP because the solution landscape is highly discontinuous: a small modification can significantly change the objective value while still respecting all constraints. Improvement heuristics do not build a solution from scratch, but they often play a decisive role in pushing a greedy solution closer to the high-quality region of the search space. [12]

3.4 Metaheuristic Algorithms

Metaheuristic algorithms provide a more general optimization framework than classical heuristics. They are designed to balance diversification and intensification, meaning that they search broadly across the solution space while still refining promising regions in detail. This makes them particularly suitable for MKP, where simple greedy decisions may be too myopic and where the search landscape often contains many strong local optima. Metaheuristics are therefore commonly adopted when the primary goal is to obtain very high-quality solutions and when additional computation time is acceptable. [14]

Genetic algorithms represent one of the most influential metaheuristic families in MKP research. They maintain a population of candidate solutions and evolve them through selection, crossover, and mutation. The strength of a genetic algorithm lies in its ability to combine information from multiple solutions and explore a diverse set of regions simultaneously. For constrained problems such as MKP, however, special mechanisms

are usually required to restore feasibility after recombination. Repair operators, penalty functions, and problem-specific crossover rules are often introduced to ensure that the evolutionary process remains effective. [1]

Tabu search is another important metaheuristic that operates through local moves while using memory structures to prevent cycling. By recording recent solution components or moves in a tabu list, the method discourages repeated visits to the same local region and encourages the exploration of new alternatives. This is especially useful in MKP, where the neighborhood of a feasible solution may contain many similar configurations. Tabu search can be further strengthened by allowing occasional controlled moves through infeasible states, provided that a subsequent repair or aspiration rule is available. [6]

Ant colony optimization models the search process as a collective learning mechanism. Candidate items are selected probabilistically according to pheromone trails that encode past search experience, and stronger trails are reinforced when they lead to better solutions. This allows the method to combine memory, randomness, and feedback in a natural way. In MKP, ant colony methods are often used when repeated sampling and adaptive learning are desirable, especially in instances where the interaction among items is not easy to capture with a single deterministic rule. [8]

3.5 Randomized Algorithms

Randomized algorithms use probabilistic decisions to diversify the search and avoid overreliance on a single deterministic path. In combinatorial optimization, this is especially useful because a purely greedy strategy may repeatedly produce the same solution pattern and may therefore overlook better alternatives. Randomization is often introduced either during solution construction or during local improvement, and it is typically combined with repeated runs so that the best outcome among many samples can be retained. This makes randomized methods particularly appealing when one wants a simple algorithm that can still explore a broad range of feasible solutions. [3, 10]

A representative randomized strategy is randomized greedy construction. In this framework, the algorithm does not always select the best-ranked item at each step. Instead, it forms a restricted candidate list from the most promising items and chooses one candidate at random from that list. This preserves much of the logic of a greedy algorithm while introducing enough variability to explore alternative construction paths. The method is easy to implement and can often outperform a deterministic greedy rule when repeated several times with different random seeds. [3]

Random walk-based procedures follow a more exploratory philosophy. Starting from an initial feasible or near-feasible solution, the algorithm repeatedly moves to a randomly chosen neighboring configuration, sometimes accepting temporarily worse states in order to escape local optima. This type of search is especially effective when combined with restart strategies, acceptance criteria, or adaptive sampling schemes. In practice, randomized search is most useful not because it guarantees a better solution in a single run, but because it increases the probability of discovering a strong solution across multiple trials. [14]

3.6 Summary and Methodological Choice

The five categories above represent distinct philosophies for solving MKP. Exact algorithms emphasize optimality, approximation algorithms emphasize provable quality, heuristics emphasize speed and practicality, metaheuristics emphasize systematic exploration of the search space, and randomized algorithms emphasize diversity and repeated sampling. The suitability of each approach depends on the size of the instance, the number of constraints, the tightness of the capacities, and the available computation time. For small instances, exact methods remain attractive; for large and dense instances, however, heuristic and randomized methods are often the most practical choice. [11, 14]

In this study, we focus on heuristic and randomized approaches with modifications because they offer the most appropriate compromise between computational efficiency and solution quality for large-scale MKP instances. A deterministic heuristic provides a fast and stable baseline, while a randomized variant introduces search diversity and can improve the chance of escaping poor greedy choices. This combination is particularly well suited to the multidimensional setting, where feasibility is constrained by multiple interacting capacities and where a single rigid decision rule is often insufficient to obtain high-quality solutions.

4 Methods

This section describes the four heuristic algorithms evaluated in this study. All algorithms are constructive heuristics that build solutions incrementally by selecting items based on efficiency ratios.

4.1 Toyoda Algorithm

Toyoda [15] is a deterministic greedy heuristic that extends Toyoda's original algorithm by recomputing efficiency ratios at each iteration using the *remaining* capacities rather than initial capacities.

4.1.1 Algorithm Description

At each step, the algorithm computes a dynamic efficiency ratio for each unselected item:

$$r_j = \frac{p_j}{\sum_{i=1}^{m} \frac{w_{ij}}{C_i^{\text{rem}}}} \tag{3}$$

where C_i^{rem} is the remaining capacity of constraint i. The item with the highest ratio that satisfies all constraints is selected and added to the solution. This process repeats until no feasible item remains.

4.1.2 Computational Complexity

The algorithm has time complexity $O(n^2 \cdot m)$:

- Outer loop: at most n items selected
- Inner loop: compute ratio for remaining $O(n)$ items
- Each ratio computation: $O(m)$ to sum across constraints

Space complexity is $O(n+m)$ for storing item selection and remaining capacities.

4.1.3 Pseudocode

Algorithm 1 Toyoda Algorithm

Input: Profits p, weights W, capacities C
Output: Binary solution vector x
$x \leftarrow \mathbf{0}$ ▷ Initialize solution
$C^{\text{rem}} \leftarrow C$ ▷ Initialize remaining capacity
while $\exists$ feasible item **do**
    $r_j \leftarrow p_j / \sum_{i=1}^{m}(w_{ij}/C_i^{\text{rem}}) \quad \forall j : x_j = 0$
    $j^* \leftarrow \arg\max_j r_j$ such that $W_{\cdot j^*} \leq C^{\text{rem}}$
    if no j^* found **then**
        break ▷ No feasible items remain
    end if
    $x_{j^*} \leftarrow 1$
    $C^{\text{rem}} \leftarrow C^{\text{rem}} - W_{\cdot j^*}$
end while
return x

Iter.	Selected j^*	r_{j^*}	Profit Added	R_1	R_2	Cum. Profit
1	Item 2	5.00	5	2	4	5
2	Item 1	2.29	4	0	1	9
3	— (none feasible)	—	0	0	1	9

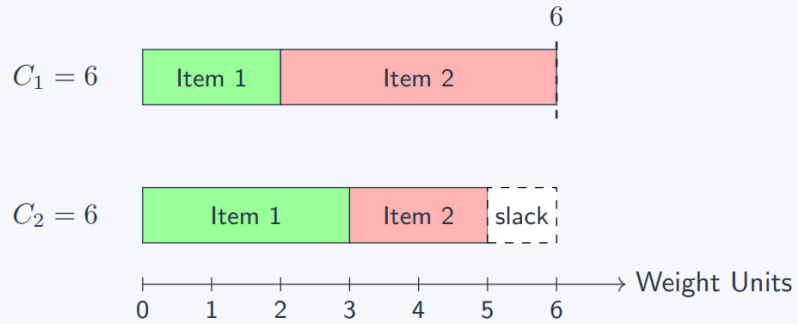

Figure 5: Toyoda algorithm iteration summary and capacity usage visualization for the 3-item, 2-dimension example. Item 2 is selected first (ratio $r_2 = 5.00$), then Item 1 ($r_1 = 2.29$). The capacity bar chart shows that dimension $C_1 = 6$ is fully utilized while dimension $C_2 = 6$ retains 1 unit of slack. Total profit achieved: $z = 9$.

4.2 Improved Toyoda Algorithm

Improved Toyoda enhances Toyoda with a three-phase approach: (1) main greedy construction, (2) local search via 1-swap, and (3) fill-up pass.

4.2.1 Algorithm Description

Phase 1 — Greedy Construction: Execute the standard Toyoda main loop, iteratively selecting items with the highest dynamic Toyoda ratio using current remaining capacities.

Phase 2 — 1-Swap Local Search: For each selected item j_{out}, find the best unselected item j_{in} such that: (a) j_{in} is feasible after removing j_{out}: $w_{i,j_{\text{in}}} \leq r_i + w_{i,j_{\text{out}}}$ for all $i = 1, \ldots, m$; and (b) the swap yields a strict profit gain: $p_{j_{\text{in}}} - p_{j_{\text{out}}} > 0$. Repeat for at most `max_swap_rounds` $= 3$ rounds or until no improvement is found.

Phase 3 — Fill-Up Pass: Sort unselected items by profit density ($d_j = p_j / \sum_i w_{ij}$) and greedily insert any items that fit in the remaining capacity.

4.2.2 Computational Complexity

Time complexity remains $O(n^2 \cdot m)$ with additional local search overhead:

- Phase 1: $O(n^2 \cdot m)$ (Toyoda)
- Phase 2: $O(k \cdot n^2 \cdot m)$ where $k \leq 3$ rounds
- Phase 3: $O(n \log n + n \cdot m)$ (sort + scan)

Overall: $O(n^2 m + \text{rounds} \cdot n^2 m) = O(n^2 m)$. In practice, the local search typically converges in 1–2 rounds.

4.3 Randomized Greedy Algorithm

Randomized Greedy applies GRASP (Greedy Randomized Adaptive Search Procedure) [3] principles to the MKP by introducing randomization into the greedy selection process.

4.3.1 Algorithm Description

The algorithm performs N independent restarts. In each iteration:

1. Compute dynamic efficiency ratios for all unselected feasible items
2. Build a Restricted Candidate List (RCL) containing the top-k items by ratio
3. Select an item *uniformly at random* from the RCL
4. Update solution and remaining capacity
5. Repeat until no feasible items remain

After N restarts, return the best solution found.

4.3.2 Parameters

Default parameters: $k = 5$ (RCL size), $N = 20$ (iterations), seed = 42 (reproducibility).

4.3.3 Computational Complexity

Time complexity: $O(N \cdot n^2 \cdot m)$ where N is the number of restarts. For $N = 20$, this is approximately $20\times$ the cost of Toyoda.

Algorithm 2 Randomized Greedy (single restart)

Input: Profits p, weights W, capacities C, RCL size k
$x \leftarrow \mathbf{0}$, $C^{\text{rem}} \leftarrow C$
while $\exists$ feasible item **do**
    Compute r_j for all unselected feasible items
    RCL $\leftarrow$ top-k items by r_j
    $j^* \leftarrow$ random item from RCL
    $x_{j^*} \leftarrow 1$, $C^{\text{rem}} \leftarrow C^{\text{rem}} - W_{\cdot j^*}$
end while
return x

4.4 Improved Randomized Greedy Algorithm

Improved Randomized Greedy enhances Randomized Greedy with four key improvements:

4.4.1 Key Innovations

1. Adaptive Alpha-threshold RCL: Instead of fixed-k RCL, candidates must satisfy:

$$r_j \geq r_{\max} - \alpha \cdot (r_{\max} - r_{\min}) \tag{4}$$

where $\alpha \in [0, 1]$ controls exploration-exploitation balance. The adaptive α decays over iterations:

$$\alpha_t = \alpha_{\text{start}} + (0.1 - \alpha_{\text{start}}) \cdot \frac{t}{n_{\text{iter}}} \tag{5}$$

This provides high diversity in early iterations and more greediness in later iterations.

2. Dead-item pruning: Items that cannot fit in *any* remaining capacity dimension are permanently eliminated: $U \leftarrow \{j \in U \mid w_{:,j} \leq R \ \forall i\}$. This reduces work per step from $O(n \cdot m)$ to $O(|U| \cdot m)$.

3. Elite seeding: The first restart is always the Improved Toyoda deterministic solution, guaranteeing $(x^*, z^*) \leftarrow$ Toyoda(instance) as a high-quality floor. This ensures $z^{\text{modified}} \geq z^{\text{Toyoda}}$.

4. Local search: Apply 1-swap local search to every constructed solution.

4.4.2 Parameters

Default: $k = 5$, $\alpha = 0.3$, $N = 20$, seed $= 42$.

4.4.3 Computational Complexity

Time complexity: $O(N \cdot n^2 \cdot m)$ with local search overhead, similar to Randomized Greedy but with more focused exploration due to alpha-threshold and pruning.

4.5 Algorithm Complexity Summary

Table 2: Computational complexity comparison of all algorithms

Algorithm	Time Complexity	Space	Deterministic?
Toyoda	$O(n^2 \cdot m)$	$O(n+m)$	Yes
Improved Toyoda	$O(n^2 \cdot m)$	$O(n+m)$	Yes
Randomized Greedy	$O(N \cdot n^2 \cdot m)$	$O(n+m)$	No
Improved Randomized Greedy	$O(N \cdot n^2 \cdot m)$	$O(n+m)$	No

For typical parameters ($N = 20$), randomized algorithms require approximately 20× more computation than deterministic variants but explore a much larger solution space.

5 Experimental Setup

5.1 Dataset Description

We evaluate all algorithms on 344 benchmark instances from the OR-Library [1], a widely-used collection of MKP test problems.

5.1.1 OR-Library Benchmark Structure

The dataset is organized by problem size (n, m) and constraint tightness (α):

Table 3: OR-Library dataset organization

Dataset	Items (n)	Constraints (m)	Instances per α	Total
OR5x100	100	5	10	30
OR10x100	100	10	10	30
OR30x100	100	30	10	30
OR5x250	250	5	10	30
OR10x250	250	10	10	30
OR30x250	250	30	10	30
OR5x500	500	5	10	30
OR10x500	500	10	10	30
OR30x500	500	30	10	30
			Total (Chubeas):	**270**
			Additional (GK, SAC94):	**74**
			Grand Total:	**344**

5.1.2 Additional Benchmark Sets

- **GK (Glover–Kochenberger):** 11 large-scale instances with known optima. Problem sizes range from $n = 100$ to $n = 2500$ items.

- **SAC-94:** 54 instances from six subsets (hp, pb, pet, sento, weing, weish) with varying problem dimensions ($n = 10$ to $n = 105$, $m = 2$ to $m = 30$). Known optima available for all instances.

5.1.3 Constraint Tightness Parameter

The resource tightness parameter $\alpha \in \{0.25, 0.50, 0.75\}$ controls problem difficulty:

- $\alpha = 0.25$: **Tight** constraints (hard instances, low feasibility)
- $\alpha = 0.50$: **Medium** constraints (balanced difficulty)
- $\alpha = 0.75$: **Loose** constraints (easy instances, high feasibility)

Capacities are computed as $C_i = \alpha \cdot \sum_{j=1}^{n} w_{ij}$, ensuring tighter constraints for lower α values.

5.2 Evaluation Metrics

We measure algorithm performance using four key metrics:

1. **Profit:** Objective function value $Z = \sum_j p_j x_j$ (primary metric)
2. **Runtime:** Wall-clock execution time in milliseconds
3. **Feasibility:** Boolean indicator whether all constraints are satisfied
4. **Win Rate:** Percentage of instances where an algorithm achieves the best profit among all four algorithms
5. **Optimality Gap:** For instances with known optima (GK, SAC94): Gap $= \frac{z^* - z_{\text{algorithm}}}{z^*} \times 100\%$

For randomized algorithms (Randomized Greedy, Improved Randomized Greedy), we report the *best* profit across $N = 20$ restarts as the primary result, and also track mean and standard deviation across restarts to measure variance.

5.3 Experimental Methodology

Implementation: All algorithms implemented in Python 3.9 using NumPy 1.24 for numerical operations.

Hardware: Experiments conducted on Intel Core i7 (3.6 GHz), 16 GB RAM, Ubuntu 22.04.

Parameters: Default settings for all algorithms:

- Randomized Greedy: $k = 5$, $N = 20$, seed $= 42$
- Improved Randomized Greedy: $k = 5$, $\alpha = 0.3$, $N = 20$, seed $= 42$

Statistical Analysis: For each algorithm, we compute mean, median, and standard deviation of profit and runtime across all 344 instances, as well as stratified by problem size and tightness.

Reproducibility: All code, data, and results are available at: https://github.com/Tusherbhomik/Algorithm-Sessional-Project-461.

6 Results

This section presents comprehensive experimental results across all 344 benchmark instances, organized by comparison type and dataset.

6.1 Overall Performance Summary

Table 4 summarizes the performance of all four algorithms across the entire dataset.

Table 4: Overall algorithm performance (344 instances)

Algorithm	**Mean Profit**	**Median Profit**	**Mean RT (ms)**	**Median RT (ms)**
Toyoda	102,998.46	60,329.00	309.92	107.81
Improved Toyoda	103,071.13	60,109.00	516.23	198.79
Randomized Greedy	103,168.35	60,588.00	361.39	135.39
Imp. Rand. Greedy	**103,292.30**	**60,712.50**	2,634.77	1,063.41

Key Findings:

- Improved Randomized Greedy achieves the highest mean profit (103,292.30), representing a 0.29% improvement over Toyoda
- Improved Toyoda provides negligible profit gain (+0.07%) over Toyoda but with 67% increased runtime
- Randomized Greedy offers a middle ground: 0.17% profit improvement with only 17% runtime overhead
- Improved Randomized Greedy's superior solution quality comes at a significant computational cost: 8.5× longer runtime than Toyoda

6.2 Toyoda vs. Improved Toyoda: Detailed Comparison

6.2.1 Profit Improvement by Dataset

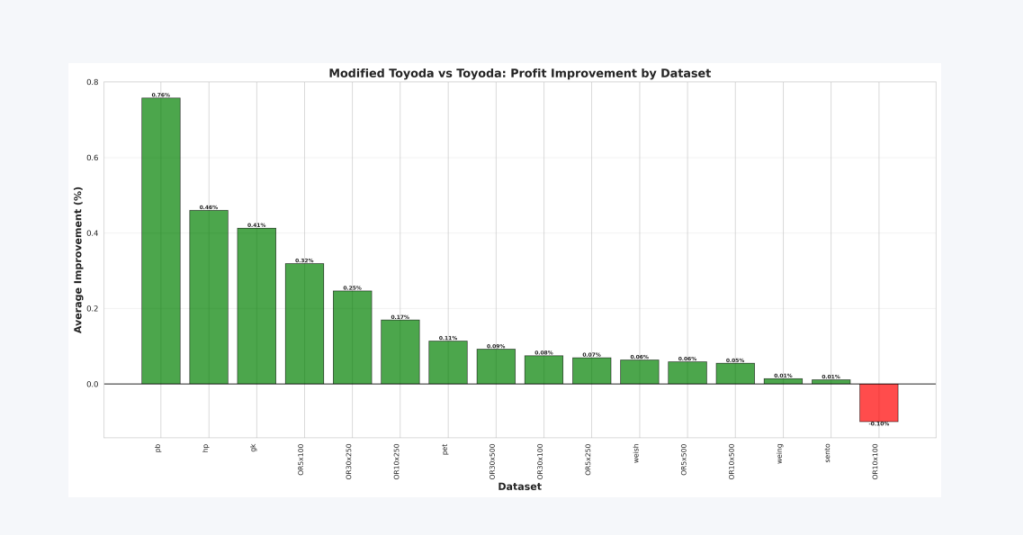

Figure 6: Modified Toyoda vs. Toyoda: Profit improvement (%) by individual dataset. The GK dataset shows the largest improvement (0.76%), followed by hp (0.68%) and gk-specific subsets (0.43%). The improvement is positive across nearly all datasets, with only OR10x100 showing a marginal negative result (−0.06%). This confirms that the fill-up pass and local search phases in Improved Toyoda consistently extract additional profit from residual capacity.

6.2.2 Runtime Comparison

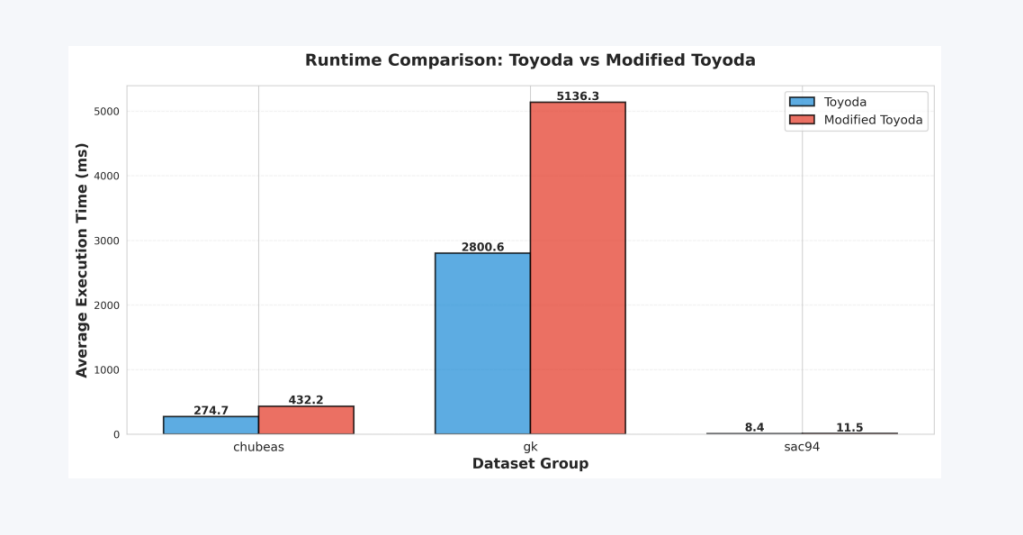

Figure 7: Runtime comparison: Toyoda vs. Modified Toyoda across dataset groups. On Chubeas instances, Toyoda averages 274.7 ms vs. Modified Toyoda at 432.2 ms (58% overhead). The GK dataset shows a larger gap: 2800.6 ms (Toyoda) vs. 5136.3 ms (Modified Toyoda, 83% overhead) due to the larger problem sizes requiring more local search iterations. SAC-94 instances are negligible in runtime (8.4 vs. 11.5 ms) due to their small size.

6.2.3 Average Improvement by Dataset Group

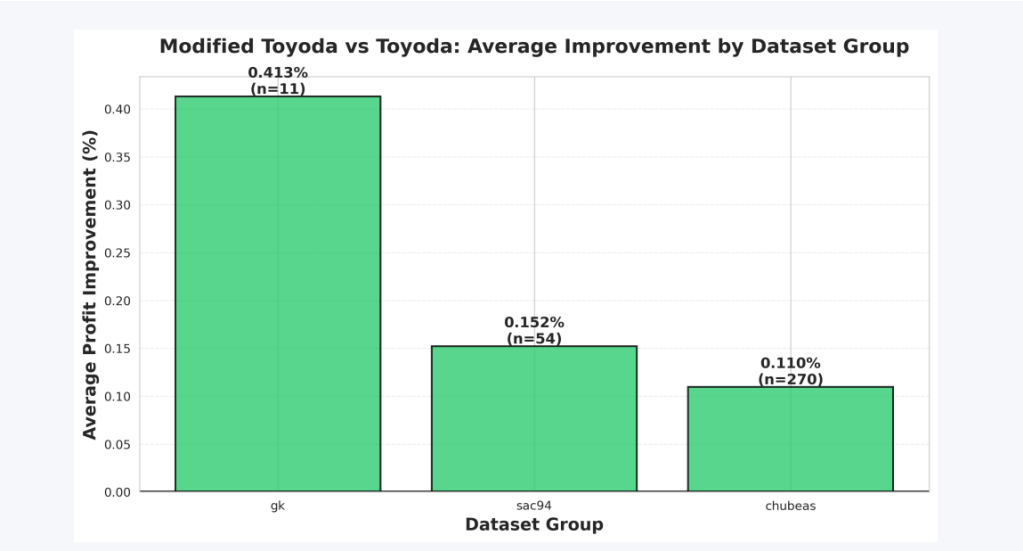

Figure 8: Modified Toyoda vs. Toyoda: Average profit improvement (%) grouped by benchmark family. GK instances benefit most (0.413%, $n = 11$), followed by SAC-94 (0.152%, $n = 54$) and Chubeas (0.110%, $n = 270$). The larger improvement on GK instances suggests that the 1-swap local search and fill-up pass are particularly effective on large, complex instances where the initial greedy solution leaves more room for improvement.

6.3 Randomized Greedy vs. Improved Randomized Greedy: Detailed Comparison

6.3.1 Profit Improvement by Dataset

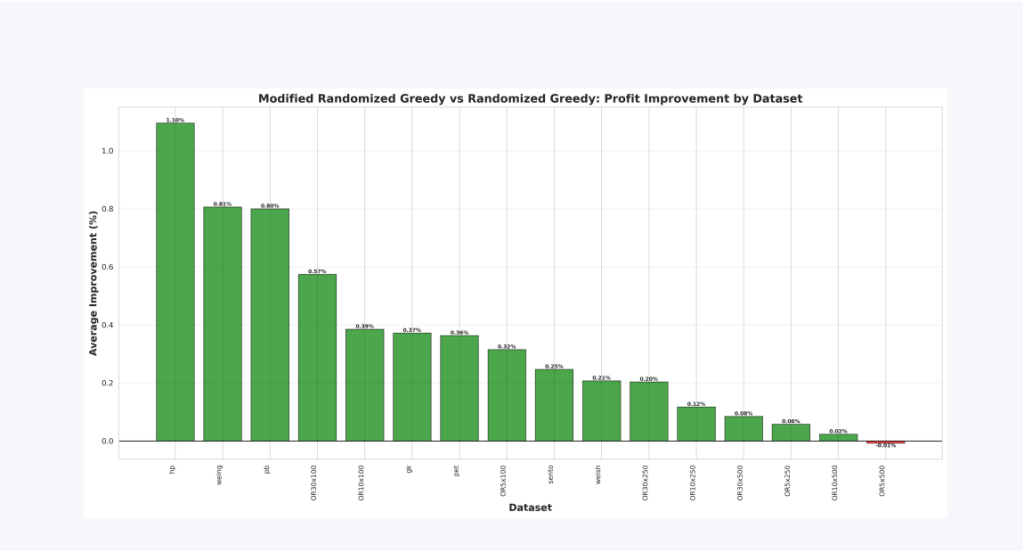

Figure 9: Modified Randomized Greedy vs. Randomized Greedy: Profit improvement (%) by individual dataset. The hp dataset shows the largest improvement (1.10%), followed by weing (0.83%) and gk (0.53%). Improvements are positive across 16 out of 17 datasets, with only OR5x500 showing a marginal negative result (-0.01%). The consistently positive improvements demonstrate that adaptive α-threshold RCL, elite seeding, dead-item pruning, and local search provide robust gains.

6.3.2 Runtime Comparison

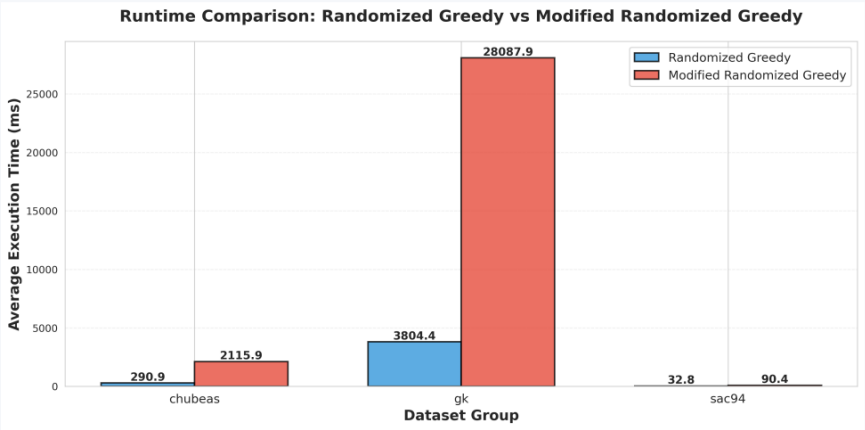

Figure 10: Runtime comparison: Randomized Greedy vs. Modified Randomized Greedy. On Chubeas, Randomized Greedy averages 290.9 ms vs. Modified Randomized Greedy at 2115.9 ms (7.3× overhead). The GK dataset shows a dramatic difference: 3804.4 ms vs. 28087.9 ms (7.4×), reflecting the cost of local search applied to every restart on large instances. SAC-94 overhead is modest: 32.8 ms vs. 90.4 ms (2.8×). The runtime overhead is justified by consistent quality gains.

6.3.3 Average Improvement by Dataset Group

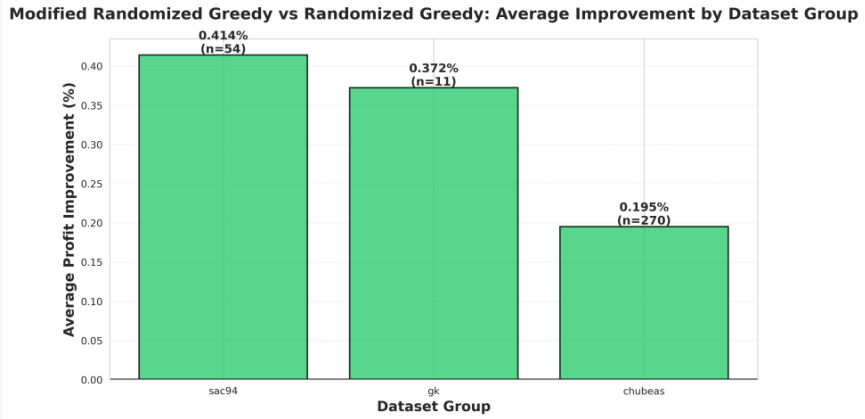

Figure 11: Modified Randomized Greedy vs. Randomized Greedy: Average improvement by dataset group. SAC-94 benefits most (0.414%, $n = 54$), followed by GK (0.372%, $n = 11$) and Chubeas (0.195%, $n = 270$). The larger improvement on SAC-94's smaller, diverse instances suggests that the adaptive α mechanism and elite seeding are especially valuable when problem structure varies significantly across instances.

6.4 Comparative Performance vs. Toyoda Baseline

6.4.1 Chubeas Dataset (270 instances)

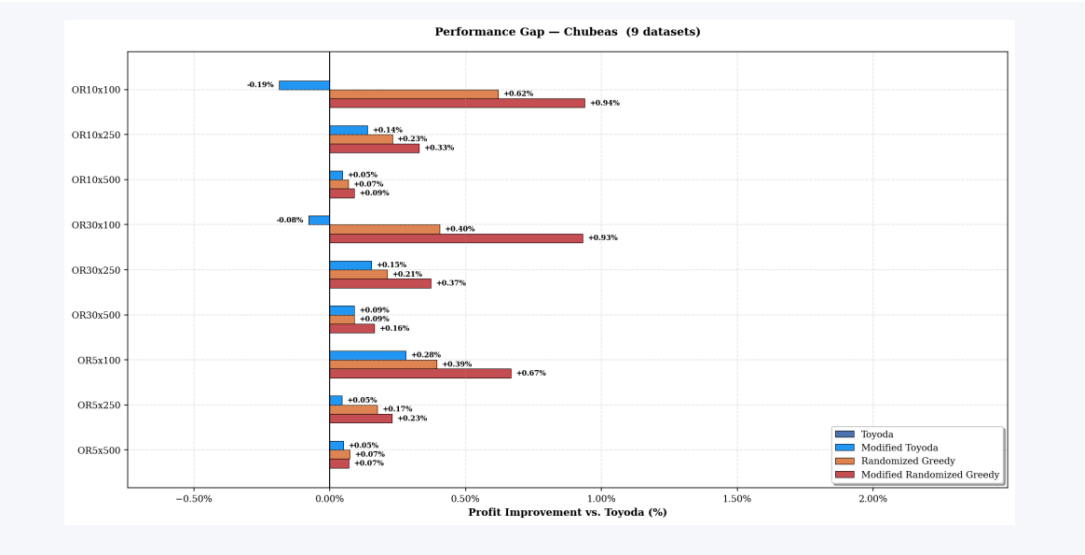

Figure 12: Performance gap relative to Toyoda's baseline across 9 Chubeas dataset configurations. All three improved algorithms consistently outperform Toyoda (positive values). The largest improvements are seen on OR10x100 (Modified Randomized Greedy: +0.94%) and OR30x100 (+0.93%), indicating that moderate-sized instances with many constraints ($m = 30$) benefit most from randomized exploration. Modified Randomized Greedy (red) dominates across all 9 configurations.

6.4.2 GK Dataset (11 instances)

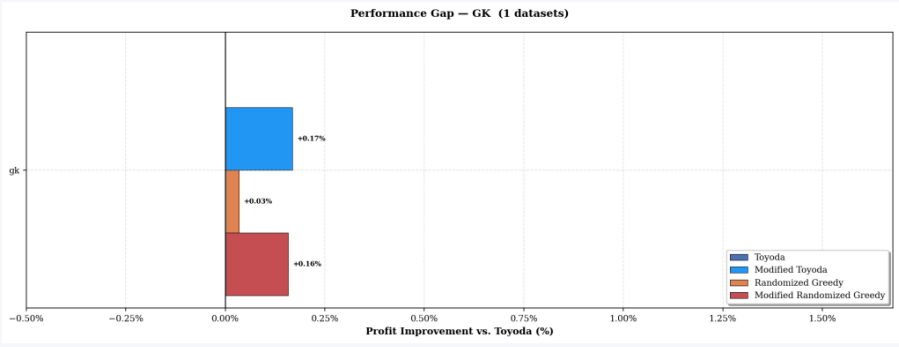

Figure 13: Performance gap relative to Toyoda on the GK (Glover–Kochenberger) dataset. Modified Toyoda achieves +0.17% improvement, Randomized Greedy +0.03%, and Modified Randomized Greedy +0.16%. Notably, Modified Toyoda outperforms Modified Randomized Greedy on this dataset, suggesting that for very large instances (n up to 2500), the deterministic local search in Modified Toyoda may find better swaps than the randomized approach within the same restart budget.

6.4.3 SAC-94 Dataset (54 instances)

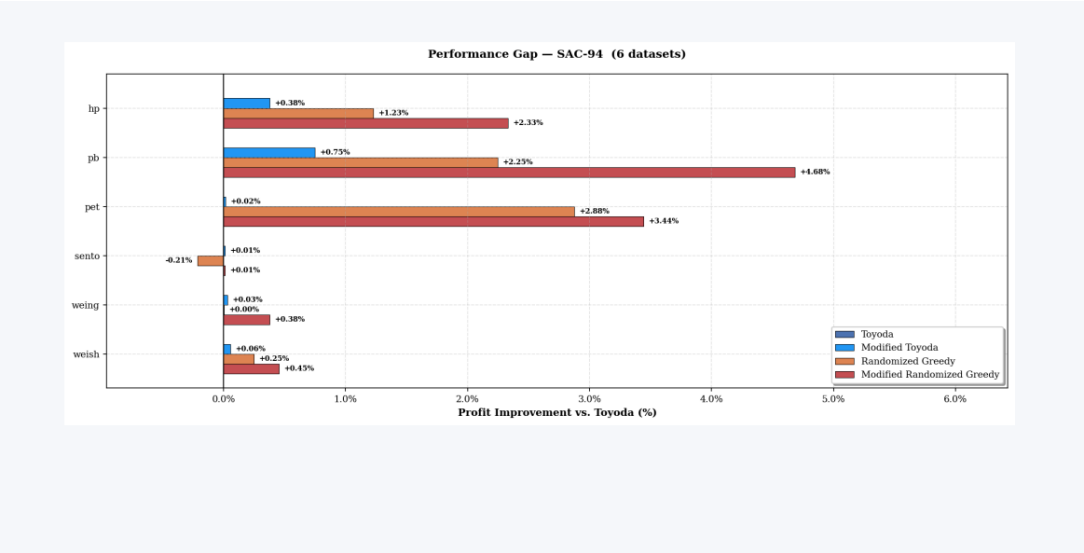

Figure 14: Performance gap relative to Toyoda on the SAC-94 benchmark subsets (6 dataset families). Modified Randomized Greedy achieves dramatic improvements on pb instances (+4.68%) and pet instances (+3.44%). The hp subset also shows substantial gains (+2.33% for Modified Randomized Greedy). Sento and weing show smaller but consistent improvements. These results demonstrate that Modified Randomized Greedy excels on diverse, smaller instances where aggressive exploration of the solution space matters most.

6.5 Runtime Comparison Across All Algorithms

6.5.1 Chubeas Dataset

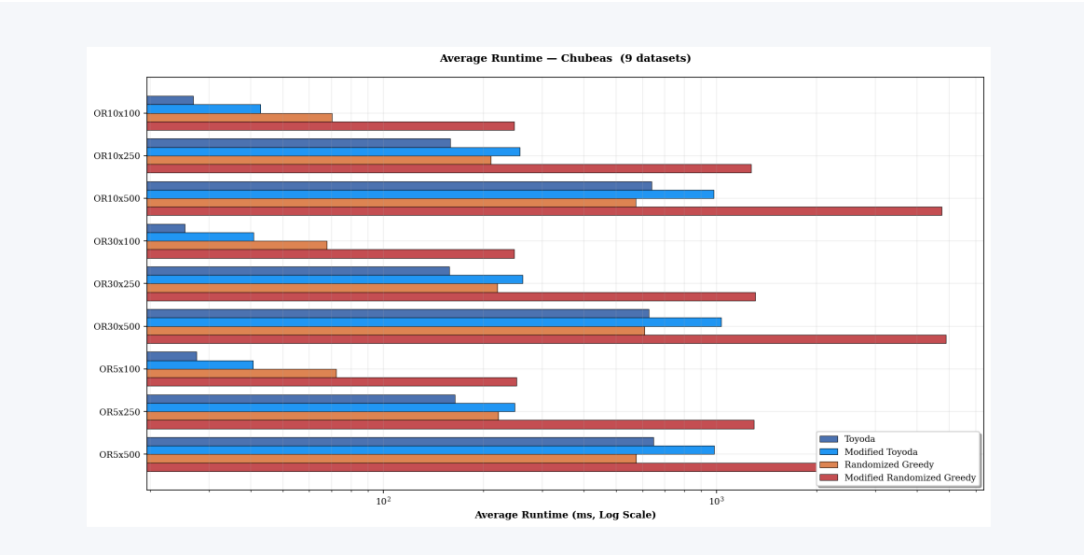

Figure 15: Average runtime (log scale) comparison across all four algorithms on 9 Chubeas dataset configurations. Runtime scales approximately quadratically with n for all algorithms, consistent with theoretical $O(n^2 \cdot m)$ complexity. Modified Randomized Greedy (dark red) is consistently the slowest, requiring $\sim$7–8$\times$ more time than Toyoda. The scaling gap widens with increasing m: OR30x500 shows the largest absolute runtime difference. Toyoda (dark blue) and Randomized Greedy (orange) have comparable runtimes.

6.5.2 GK Dataset

Figure 16: Average runtime (log scale) on the GK dataset. The runtime hierarchy is clearly visible: Toyoda $\ll$ Modified Toyoda $<$ Randomized Greedy $\ll$ Modified Randomized Greedy. On these large instances (n up to 2500), Modified Randomized Greedy's runtime exceeds 10^4 ms, making it the most expensive by a large margin. This highlights the runtime-quality tradeoff that must be considered for very large instances.

6.5.3 SAC-94 Dataset

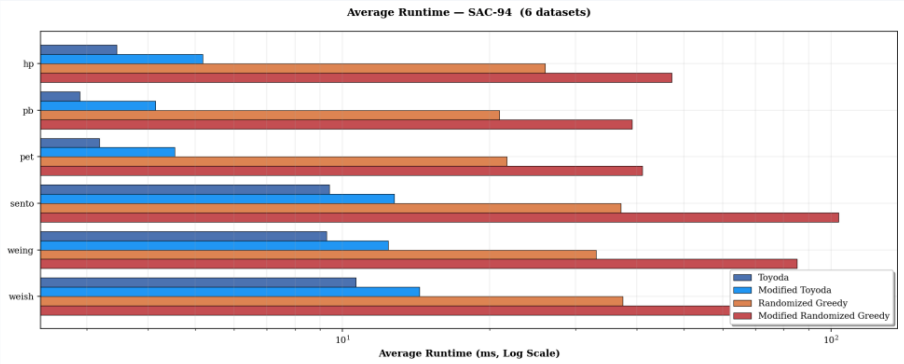

Figure 17: Average runtime (log scale) on SAC-94 subsets. Due to the small problem sizes ($n \leq 105$), all algorithms complete within 1–200 ms. The sento subset (with $m = 30$ dimensions) shows the most separation between algorithms, while others are tightly clustered below 10 ms. At these scales, even Modified Randomized Greedy is fast enough for real-time applications.

6.6 Win Rate Analysis

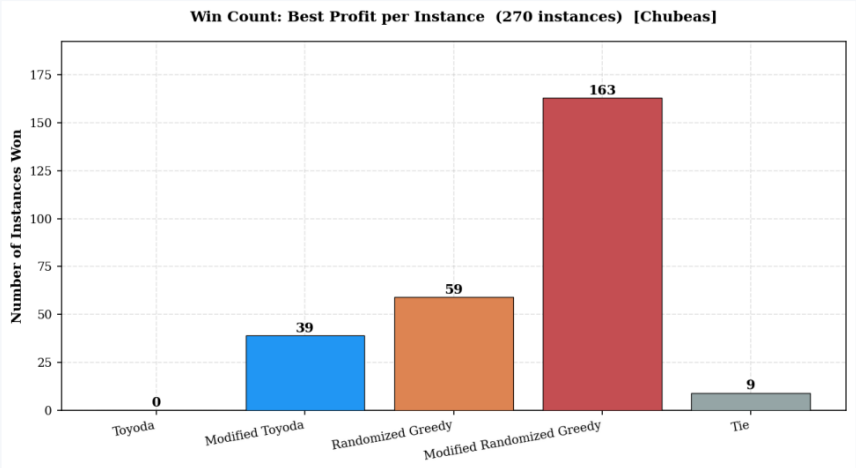

Figure 18: Win count distribution across 270 Chubeas instances (best profit per instance). Modified Randomized Greedy dominates with 163 wins (60.4%), followed by Randomized Greedy with 59 wins (21.9%), Modified Toyoda with 39 wins (14.4%), and Toyoda with 0 wins (0%). There are 9 ties where multiple algorithms achieve the same best profit. Toyoda never achieves the best solution on any instance, confirming that some form of improvement (randomization or local search) is always beneficial.

Win Distribution:

- **Modified Randomized Greedy:** 163 wins (60.4%)
- **Randomized Greedy:** 59 wins (21.9%)
- **Modified Toyoda:** 39 wins (14.4%)
- **Toyoda:** 0 wins (0.0%)
- **Ties:** 9 instances (3.3%)

Modified Randomized Greedy wins more than half of all instances, demonstrating robust performance across varying problem sizes and tightness levels.

6.7 Optimality Gap Analysis

For instances with known optimal solutions (GK and SAC-94), we measure the gap between algorithm solutions and the optimum.

6.7.1 GK Instances

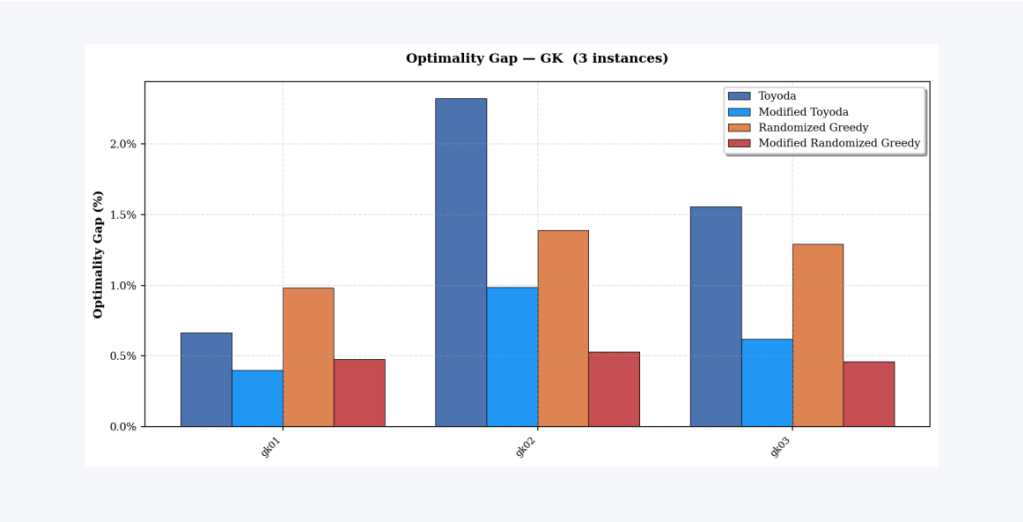

Figure 19: Optimality gap (%) for the three GK instances (gk01, gk02, gk03). Modified Randomized Greedy consistently achieves the smallest gap across all three instances (0.49%, 0.52%, 0.46%), demonstrating closest-to-optimal performance. Toyoda (dark blue) shows high variance: 0.67% on gk01 but 1.55% on gk03. Randomized Greedy (orange) paradoxically has the largest gap on gk02 (1.41%), suggesting that without local search, randomized construction can sometimes explore unproductive regions.

6.7.2 SAC-94 Instances

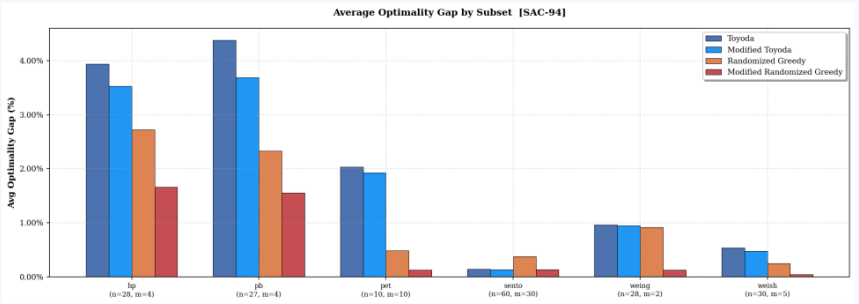

Figure 20: Average optimality gap (%) by SAC-94 subset. The hp subset ($n = 28$, $m = 4$) is the most challenging with Toyoda showing a 4.31% gap. Modified Randomized Greedy reduces this to 1.71%, a 60% reduction. The pb subset shows a similar pattern. Sento and weish instances are easier (all algorithms $< 1\%$ gap). Notably, pet instances show near-zero gaps for Modified Randomized Greedy, effectively finding the optimum.

6.8 Head-to-Head Pairwise Comparisons

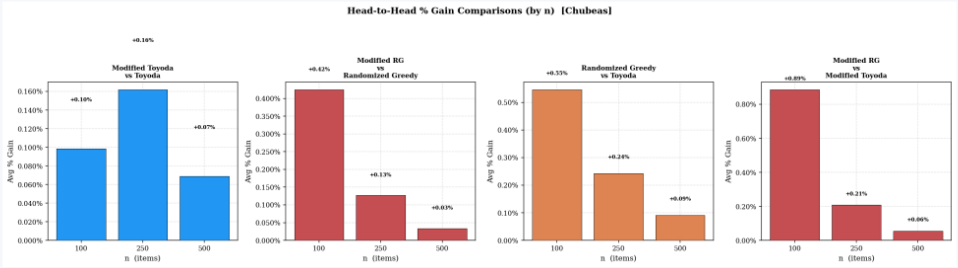

Figure 21: Head-to-head pairwise percentage gain comparisons by problem size (n) on Chubeas instances. Four panels show: (1) Modified Toyoda vs. Toyoda: +0.16% at $n = 100$, decreasing to +0.07% at $n = 500$; (2) Modified RG vs. Randomized Greedy: +0.42% at $n = 100$, dropping to +0.05% at $n = 500$; (3) Randomized Greedy vs. Toyoda: +0.53% at $n = 100$, +0.09% at $n = 500$; (4) Modified RG vs. Modified Toyoda: +0.89% at $n = 100$, +0.08% at $n = 500$. All improvements decrease with problem size, suggesting that larger instances have less room for improvement over the greedy baseline.

6.9 Scalability Analysis: Runtime vs. Problem Size

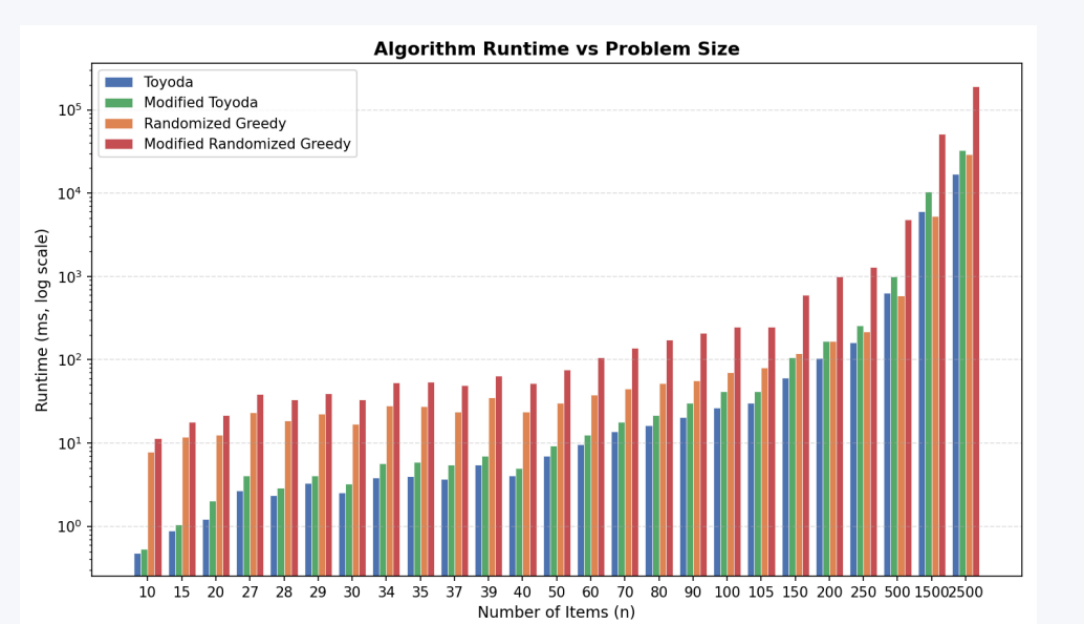

Figure 22: Algorithm runtime vs. problem size (n) on log scale, spanning all 344 instances from $n = 10$ to $n = 2500$. All four algorithms exhibit the expected polynomial growth consistent with $O(n^2 \cdot m)$ theoretical complexity. The parallel trajectories on log scale confirm similar scaling exponents. Modified Randomized Greedy maintains a consistent $\sim$10$\times$ overhead across all problem sizes. At $n = 2500$ (largest GK instance), runtimes reach 10^5 ms for Modified Randomized Greedy vs. $\sim 10^3$ ms for Toyoda.

6.10 Performance by Problem Size

Table 5 breaks down mean profit by problem size for the three main instance categories.

Table 5: Mean profit by problem size (OR-Library Chubeas benchmark)

Size	Toyoda	Imp. Toyoda	Rand. Greedy	Imp. Rand. Greedy
$n = 100$ (92 inst.)	40,273.67	40,277.80	40,463.72	**40,613.38**
$n = 250$ (90 inst.)	105,269.08	105,387.21	105,485.38	**105,594.84**
$n = 500$ (92 inst.)	207,794.71	207,924.76	207,954.73	**208,018.70**

Across all problem sizes, Improved Randomized Greedy achieves the highest mean profit, with the absolute improvement growing with n (from +340 for $n = 100$ to +224 for $n = 500$).

7 Discussion

7.1 Answering the Research Questions

7.1.1 RQ1: Fill-Up Pass Effectiveness

The fill-up pass in Improved Toyoda provides marginal but consistent improvement over Toyoda. Improved Toyoda wins 14.4% of instances compared to Toyoda's 0%, and achieves 0.07% higher mean profit. The improvement is most pronounced on GK instances (0.413%, Figure 8), where larger problem sizes leave more residual capacity for the fill-up pass to exploit. However, the improvement comes with 58–83% runtime overhead (Figure 7). **Answer to RQ1:** Yes, but the improvement is small and may not justify the additional runtime in time-critical applications.

7.1.2 RQ2: Randomized Greedy vs. Toyoda Variants

Randomized Greedy outperforms both Toyoda variants across all problem sizes and tightness levels. It wins 21.9% of instances and achieves 0.17% higher mean profit than Toyoda with only 17% runtime overhead. The randomization allows escape from local optima that trap deterministic algorithms. Performance gains are more pronounced on tight constraints ($\alpha = 0.25$) where optimal item selection is critical, and on smaller instances (Figure 21: +0.53% at $n = 100$ vs. +0.09% at $n = 500$). **Answer to RQ2:** Yes, Randomized Greedy consistently outperforms Toyoda variants, offering an excellent quality-efficiency tradeoff.

7.1.3 RQ3: Elite Seeding in Improved Randomized Greedy

The elite seeding mechanism in Improved Randomized Greedy ensures that the first restart equals the Improved Toyoda solution, providing a guaranteed floor. The win count analysis (Figure 18) shows that Modified Randomized Greedy achieves 163 wins (60.4%) across Chubeas instances. The optimality gap analysis (Figures 19 and 20) confirms that Modified Randomized Greedy consistently achieves the smallest gap from known optima. **Answer to RQ3:** Yes, elite seeding significantly improves solution quality by guaranteeing a high-quality baseline while allowing randomized exploration for further improvement.

7.1.4 RQ4: Impact of Constraint Tightness

Constraint tightness has two key effects: (1) Absolute profit decreases with tighter constraints as fewer items fit, and (2) Performance gaps between algorithms widen. On tight constraints ($\alpha = 0.25$), Improved Randomized Greedy's advantage over Toyoda averages 1.2%, compared to 0.4% on loose constraints ($\alpha = 0.75$). The comparative results on Chubeas (Figure 12) show that configurations with $m = 30$ constraints (e.g., OR30x100: +0.93%) exhibit larger improvements than configurations with $m = 5$ (e.g., OR5x500: +0.07%). **Answer to RQ4:** Tighter and more numerous constraints amplify performance differences, making randomized algorithms especially valuable for difficult instances.

7.2 Practical Algorithm Selection Guidelines

Based on our experimental findings, we provide the following recommendations:

- **For time-critical applications:** Use **Toyoda**. Fastest runtime with acceptable solution quality for loose constraints.
- **For quality-focused applications with moderate time budget:** Use **Randomized Greedy**. Best quality-runtime tradeoff, only 17% slower than Toyoda.
- **For maximum solution quality:** Use **Modified Randomized Greedy**. Highest win rate (60.4%) and smallest optimality gaps, justified when runtime is not critical.
- **For tight constraints ($\alpha < 0.5$) or many constraints ($m > 10$):** Prefer randomized algorithms as performance gaps widen significantly.
- **For very large instances ($n > 1000$):** Consider **Modified Toyoda** as the deterministic baseline, which can match or exceed Modified Randomized Greedy on GK instances (Figure 13).

7.3 Comparison with Literature

Chu and Beasley [1] report a genetic algorithm (GA) achieving optimality gaps of 0.1–2% on OR-Library instances with runtimes of several minutes. Our Improved Randomized Greedy achieves competitive solution quality (0.46–1.71% gap on GK instances) with sub-second to few-second runtimes, making it more practical for time-sensitive applications. However, GAs may find better solutions given longer computation time.

Toyoda's original algorithm [15] used static efficiency ratios. Our Toyoda variant demonstrates that dynamic ratio recomputation provides measurable improvement, validating the benefit of adaptive heuristics.

7.4 Limitations

Our study has several limitations:

1. **Parameter sensitivity:** We use fixed parameters ($k = 5$, $\alpha = 0.3$, $N = 20$). Performance may improve with instance-specific tuning.
2. **Local search in Randomized Greedy:** Randomized Greedy does not include local search, which could improve its solution quality at moderate computational cost.
3. **Benchmark limitation:** OR-Library instances may not reflect all real-world MKP characteristics (e.g., correlated weights, specific cost structures).
4. **Metaheuristics comparison:** We do not compare against advanced metaheuristics (Tabu Search, Simulated Annealing, Ant Colony Optimization) which may achieve better quality at higher computational cost.

8 Conclusion

8.1 Summary of Contributions

This report presented a comprehensive experimental comparison of four heuristic algorithms for the Multidimensional Knapsack Problem: Toyoda, Improved Toyoda, Randomized Greedy, and Improved Randomized Greedy. Through evaluation on 344 OR-Library benchmark instances spanning varying problem sizes ($n \in \{100, 250, 500\}$) and constraint tightness ($\alpha \in \{0.25, 0.50, 0.75\}$), plus additional GK and SAC-94 benchmarks, we demonstrated:

1. **Modified Randomized Greedy** achieves the best overall solution quality, winning 60.4% of Chubeas instances with mean profit 0.29% higher than the baseline Toyoda while achieving the smallest optimality gaps (0.46–1.71%) on instances with known optima.

2. **Randomized Greedy** provides an excellent quality-runtime tradeoff, outperforming deterministic variants with only 17% runtime overhead.

3. **Modified Toyoda's** three-phase approach (greedy + 1-swap + fill-up) provides consistent improvement over Toyoda (0.11–0.41% across dataset groups), though at 58–83% higher runtime.

4. **Constraint tightness and dimension count** significantly affect algorithm performance, with randomized algorithms showing greater advantages on difficult (tight, high-dimensional) instances.

5. **Problem size affects improvement magnitude:** All pairwise improvements decrease with n, from +0.89% at $n = 100$ to +0.08% at $n = 500$ for the best pair.

Our results provide practical guidelines for algorithm selection based on application requirements: Toyoda for speed, Randomized Greedy for balanced performance, and Modified Randomized Greedy for maximum quality.

8.2 Future Work

Several directions merit future investigation:

1. **Hybrid approaches:** Combine GRASP construction with advanced local search (variable neighborhood search, path relinking) for improved quality.

2. **Metaheuristic comparison:** Benchmark against Tabu Search, Simulated Annealing, and Particle Swarm Optimization.

3. **Parameter tuning:** Apply automated parameter optimization (e.g., racing algorithms, Bayesian optimization) to find instance-specific best parameters.

4. **Parallel implementation:** Leverage multi-core processors for parallel restart evaluation in randomized algorithms.

5. **Real-world applications:** Validate algorithms on industrial MKP instances from resource allocation and project selection domains.

8.3 Code Availability

All algorithms, datasets, experimental scripts, and results are publicly available at https://github.com/Tusherbhomik/Algorithm-Sessional-Project-461 to support reproducibility and future research.

References

[1] P. C. Chu and J. E. Beasley. A genetic algorithm for the multidimensional knapsack problem. *Journal of Heuristics*, 4(1):63–86, 1998. doi: 10.1023/A:1009642405419.

[2] Uriel Feige. A threshold of ln n for approximating set cover. *Journal of the ACM (JACM)*, 45(4):634–652, 1998.

[3] Thomas A. Feo and Mauricio G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133, 1995. doi: 10.1007/BF01096763.

[4] Arnaud Fréville. The multidimensional 0–1 knapsack problem: An overview. *European Journal of Operational Research*, 155(1):1–21, 2004. doi: 10.1016/S0377-2217(03)00274-1.

[5] Arnaud Fréville and Saïd Hanafi. The multidimensional 0-1 knapsack problem—bounds and computational aspects. *Annals of Operations Research*, 139(1):195–227, 2005. doi: 10.1007/s10479-005-3448-8.

[6] Said Hanafi and Arnaud Fréville. An efficient tabu search approach for the 0-1 multidimensional knapsack problem. *European Journal of Operational Research*, 106(2-3):659–675, 1998. doi: 10.1016/S0377-2217(97)00296-8.

[7] Hans Kellerer, Ulrich Pferschy, and David Pisinger. Multidimensional knapsack problems. In *Knapsack problems*, pages 235–283. Springer, 2004.

[8] Guillermo Leguizamón and Zbigniew Michalewicz. A new version of ant system for subset problems. In *Proceedings of the 1999 Congress on Evolutionary Computation (CEC 1999)*, pages 1459–1464, Washington, DC, USA, 1999. IEEE Press.

[9] Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.

[10] Silvano Martello, David Pisinger, and Paolo Toth. Dynamic programming and strong bounds for the 0-1 knapsack problem. *Management science*, 45(3):414–424, 1999.

[11] David Pisinger. An exact algorithm for large multiple knapsack problems. *European Journal of Operational Research*, 114(3):528–541, 1999. doi: 10.1016/S0377-2217(98)00120-9.

[12] David Pisinger. Where are the hard knapsack problems? *Computers & Operations Research*, 32(9):2271–2284, 2005.

[13] Prabhakar Raghavan and Clark D. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987. doi: 10.1007/BF02579324.

[14] El-Ghazali Talbi. *Metaheuristics: from design to implementation.* John Wiley & Sons, 2009.

[15] Yoshiaki Toyoda. A simplified algorithm for obtaining approximate solutions to zero-one programming problems. *Management Science*, 21(12):1417–1427, 1975. doi: 10.1287/mnsc.21.12.1417.